DevOps(SRE) Case Study

MERN Stack & Python ETL

Nisan 2026

AWS EKS • Terraform • GitHub Actions • Docker • Prometheus / Grafana

Genel Özet

Bu rapor, bir MERN stack web uygulaması ile saatlik çalışan bir Python ETL betiğinin sıfırdan production-grade bir Kubernetes ortamına taşınma sürecini kapsamaktadır. Proje; konteynerizasyon, altyapı otomasyonu, CI/CD, güvenlik, izleme ve alarm kurulumu olmak üzere altı temel DevOps disiplinini içermektedir.

- ## Github

Proje sürecinde kullanılan ve oluşturulan docker, terraform, Kubernetes dosyaları, unit testler, her iki proje içinde oluşturulan CI ve CD süreçleri, bu süreçlerde alınan hatalar ve geliştirme geçmişi https://github.com/mustafakucukdemirci/devops-case adresinden ulaşılabilir.

- ## Aws

Proje http://k8s-mernapp-merningr-0484b880f5-58312156.us-east-1.elb.amazonaws.com/ adresinden canlı olarak ulaşılabilmektedir. Bazı ISP'lerin erişim kısıtlamaları sebebiyle siteye giriş yapabilmek için DNS ayarı(google dns kullanılabilir 8.8.8.8, 8.8.4.4) yapmak gerekebilir.

- ## Grafana

Grafana üzerinden prometheusu test ve kontrol için bu adres kullanılmalıdır. http://a9a3294639282486fbc7871cee8310e4-1178655096.us-east-1.elb.amazonaws.com/dashboards . Kullanıcı adı: admin şifre: Admin1234!

Alan	Yapı
Altyapı	Terraform IaC, S3 state, DynamoDB lock
Konteyner	Multi-stage Dockerizasyon, non-root kullanıcı tanımlaması, minimum alana ihtiyaç duyan docker image.
Güvenlik	MongoDB auth, pod securityContext, Secrets
Ölçeklendirme	HPA(CPU %70'I, Bellek kullanımı %80'I geçerse yeni podlar oluşturulur), topologySpreadConstraints
İzleme	Prometheus + Grafana, Prometheus alarmlarının e-posta ile bildirilmesi
Testler	Proje için unit testler eklenmiştir.

İçindekiler

1. Proje Kapsamı ve Gereksinimler
2. Sistem Mimarisi
3. Konteynerizasyon — Docker
4. AWS Altyapısı — Terraform
5. Kubernetes Orkestrasyon
6. CI/CD Pipeline — GitHub Actions
7. Test Stratejisi
8. Güvenlik
9. İzleme ve Alarmlama
10. Sonuç

1. Proje Kapsamı ve Gereksinimler

Proje iki bağımsız uygulamayı kapsamaktadır:

- MERN Stack Web Uygulaması: React frontend, Express.js REST API (port 5050), MongoDB veritabanı. Kullanıcılar çalışan kayıtlarını (ad, pozisyon, seviye) oluşturabilir, listeleyebilir, güncelleyebilir ve silebilir.
- Python ETL Betiği: GitHub API'sini saatlik olarak sorgulayan, yanıtı işleyip yazdıran bağımsız bir Kubernetes CronJob.

1.1 Proje İsterleri

Bu case study'de istenen isterlerin listesi şu şekildedir.

- Uygulamalar Docker konteynerleri içinde çalıştırılması
- AWS EKS üzerinde Kubernetes ile orkestre edilmesi
- GitHub Actions ile otomatik ilerleyen CI/CD pipeline kurulması
- Prometheus ve Grafana ile monitoring sağlanması
- Testlerin otomatikleştirilmesi
- Güvenlik uygulamalarının takip edilmesi
- Bildirimlerin yapılandırılması

Bütün isterler karşılanmış olup proje dosyaları içerisinde bulunmaktadır. Ek olarak, testlerin otomatikleştirilmesi için unit testler eklenmiş ve CI/CD süreçlerine entegre edilmiştir. Böylece main branch her güncellendiğinde unit testlerde tetiklenmektedir. Bu testler fail olduğunda build, cloud'a deploy edilmez ve böylece sorunlar prod environment'a ulaşmadan önlenmiş olur.

Bununla birlikte tanımlanmış olan uyarılar tetiklendiğinde minorstopproject@gmail.com mail adresi trafından mail atılarak bildirimi sağlanmaktadır.

2. Sistem Mimarisi

Tüm uygulama bileşenleri AWS üzerinde çalışmakta; Terraform ile yönetilen VPC içindeki EKS kümesine dağıtılmaktadır. Dış trafik AWS ALB Ingress Controller üzerinden pod'lara yönlendirilmektedir.

2.1 Trafik Akışı

İstemci (Browser) → ALB DNS → AWS Application Load Balancer → Kubernetes Ingress → client pod (nginx:8080). API istekleri (/record, /healthcheck) nginx tarafından server pod'una (port 5050) proxy'lenir. Server pod'u MongoDB StatefulSet'e bağlanır.

2.2 AWS Kaynak Envanteri

Kaynak	Detay
VPC	3 public + 3 private subnet, NAT Gateway, Internet Gateway
EKS Cluster	Kubernetes 1.29, managed node group (t3.medium × 2, max 4)
ECR	mern-client, mern-server, python-etl — imaj taraması aktif
ALB	AWS Load Balancer Controller, IRSA ile yetkilendirilmiş
EBS CSI Driver	MongoDB PVC için, IRSA ile yetkilendirilmiş
S3 + DynamoDB	Terraform remote state ve state lock
IAM	IRSA rolleri: ALB Controller, EBS CSI Driver
Secrets	GitHub Actions Secrets (AWS kimlik bilgileri, MongoDB parolası)

3. Konteynerizasyon — Docker

Her üç uygulama da güvenlik ve boyut açısından optimize edilmiş çok aşamalı (multi-stage) Docker imajlarıyla paketlenmiştir.

3.1 Client (React + Nginx)

Stage 1 (builder): node:18-alpine üzerinde npm ci ile bağımlılıklar kurulur, npm run build ile statik dosyalar üretilir. Stage 2 (runtime): nginx:stable-alpine'e yalnızca /app/build dizini kopyalanır. Docker imajına node_modules dahil edilmez, boyut ~25 MB kalır.

- nginx port 8080 (non-root, Linux'ta <1024 portları root gerektirir)
- Özel nginx-main.conf ile PID dosyası ve tüm temp dizinleri /tmp'ye taşındı
- readOnlyRootFilesystem için /var/cache/nginx, /var/run, /tmp, /var/log/nginx emptyDir volume
- nginx kullanıcısı UID/GID 101 (non-root)

3.2 Server (Express.js)

Stage 1 (deps): npm ci --omit=dev ile yalnızca production bağımlılıkları kurulur. Stage 2 (runtime): node:18-alpine — yalnızca node_modules ve uygulama kodu kopyalanır.

- Özel nodeuser (UID 1001) oluşturulur, root ile çalışılmaz
- HEALTHCHECK: wget -qO- http://localhost:5050/healthcheck
- .dockerignore ile tests/, node_modules, .env hariç tutulur

3.3 Python ETL

- python:3.12-slim baz imaj
- pip install --no-cache-dir ile bağımlılıklar kurulur
- Non-root kullanıcı (UID 1001)

4. AWS Altyapısı — Terraform

Tüm AWS kaynakları Terraform ile kod olarak (Infrastructure as Code) tanımlanmıştır. State dosyası S3'te saklanır, DynamoDB ile kilitlenir — böylece eş zamanlı çalışmadan kaynaklanan state bozulması önlenir.

4.1 Modül Yapısı

Modül / Dosya	Açıklama
modules/vpc	terraform-aws-modules/vpc — 3 AZ, public + private subnet
modules/eks	terraform-aws-modules/eks v20 — managed node group, IRSA
modules/ecr	3 ECR reposu + lifecycle policy (son 10 imaj)
modules/iam	IRSA rolleri: ALB Controller, EBS CSI Driver
backend.tf	S3 backend + DynamoDB lock
variables.tf	AWS bölgesi, küme adı, node tipi vb.
outputs.tf	cluster_endpoint, ECR URL'leri, kubeconfig komutu

4.2 ECR Lifecycle Policy

Her ECR reposunda lifecycle policy tanımlanmıştır: en fazla 10 'tagged' imaj tutulur, untagged imajlar 1 gün sonra silinir. Bu sayede depolama maliyeti kontrol altında tutulur.

5. Kubernetes Orkestrasyon

Tüm kaynaklar mern-app namespace'inde çalışmaktadır.

5.1 Bileşenler

Bileşen	Tür	Detay
MongoDB	StatefulSet	1 replika, 5 Gi PVC (EBS gp3), auth aktif
Server	Deployment	2 replika, HPA (CPU %70, max 5), RollingUpdate
Client	Deployment	2 replika, HPA (CPU %70, max 5), RollingUpdate
Python ETL	CronJob	Zamanlanmıştır, her saat başı çalıştırılır
Ingress	Ingress	AWS ALB, / → client:8080

5.2 RollingUpdate Stratejisi

Server ve Client deployment'larında maxUnavailable: 0, maxSurge: 1 olarak ayarlanmıştır. Bu konfigürasyon, güncelleme sırasında her zaman en az 2 pod'un hizmet verdiğini garanti ederek sıfır kesinti ile deployment sağlar.

5.3 Yüksek Erişilebilirlik

- topologySpreadConstraints: Pod'lar farklı AZ'lere dağıtılır (maxSkew: 1)
- HPA: CPU kullanımı %70'i aşınca otomatik ölçeklenir (max 5 replika)
- Liveness + Readiness probe: /health endpoint'i 8080 portunda

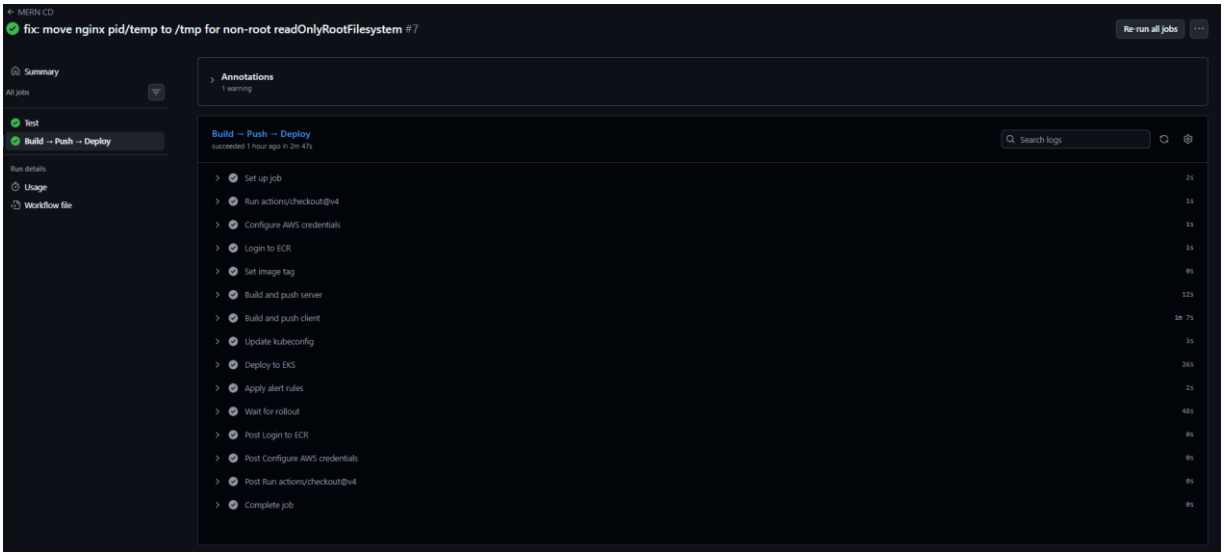
6. CI/CD Pipeline — GitHub Actions

Kod main branch'e push edildiğinde iki ayrı workflow tetiklenir: cd-mern.yml (MERN stack için) ve cd-python.yml (Python ETL için). Her workflow iki job'dan oluşur: test ve deploy.

6.1 MERN CD Pipeline

Job	Adım	Açıklama
test	MongoDB service container	mongo:7.0 ile gerçek DB bağlantısı
test	npm ci + npm test	server testleri çalıştırılır
test	Client build check	React build derlenir, hata varsa durur
deploy	AWS credentials configure	GitHub Secrets'tan kimlik bilgileri
deploy	ECR login	amazon-ecr-login action
deploy	Image tag: SHA[:8]	Her commit benzersiz tag alır
deploy	Docker build + push	Server ve client imajları ECR'a push
deploy	kubectl apply (envsubst)	YAML'lardaki değişkenler doldurulur
deploy	kubectl rollout status	120 s içinde rollout tamamlanmazsa hata

Aşağıda ki ekran görüntüsü, CI/CD pipeline akışını göstermektedir. Genel özet kısmında verilen github reposundan detaylara ulaşılabilir.



6.2 Image Tagging Stratejisi

IMAGE_TAG=${GITHUB_SHA::8} — commit SHA'nın ilk 8 karakteri kullanılır. Bu sayede her ECR imajı hangi commit'e ait olduğu izlenebilir hale gelir ve latest tag kaynaklı belirsizlik ortadan kalkar.

7. Test Stratejisi

Testler CI/CD pipeline'ının ilk aşamasını oluşturur. Herhangi bir test başarısız olursa deploy adımı hiç başlamaz.

7.1 Server Testleri (Jest + Supertest)

Test Dosyası	Test Senaryosu
record.test.mjs	GET /record — boş koleksiyon, boş dizi döner
record.test.mjs	POST /record — geçerli kayıt, 204 döner
record.test.mjs	GET /record — oluşturma sonrası 1 kayıt döner
record.test.mjs	GET /record/:id — mevcut kayıt, 200 döner
record.test.mjs	GET /record/:id — olmayan ID, 404 döner
record.test.mjs	PATCH /record/:id — güncelleme kalıcı, 200 döner
record.test.mjs	DELETE /record/:id — silme sonrası 404 döner
record.test.mjs	POST /record — boş body, 400 döner
record.test.mjs	POST /record — eksik alan (position yok), 400 döner
healthcheck.test.mjs	GET /healthcheck — 200 ve {ok: true} döner
healthcheck.test.mjs	GET /healthcheck — içerik tipi JSON

7.2 Python ETL Testleri (pytest)

Test Senaryosu
GitHub API'si çağrılır (doğru endpoint)
API yanıtı stdout'a yazdırılır
Geçersiz JSON gelirse exception fırlatılır
Ağ hatasında exception fırlatılır
API URL'si doğru formattadır

CI/CD sürecinde ki unit testlerin sonucu aşağıda gözükmektedir

```
Run server tests                                                                    2s
1  ▶ Run npm test
10
11 > server@1.0.0 test
12 > node --experimental-vm-modules node_modules/.bin/jest --forceExit
13
14 (node:2566) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
15 (Use `node --trace-warnings ...` to show where the warning was created)
16 PASS tests/record.test.mjs
17   ● Console
18
19     console.log
20       Connecting to MongoDB Atlas...
21
22       at db/conn.mjs:9:11
23
24 PASS tests/healthcheck.test.mjs
25   ● Console
26
27     console.log
28       Connecting to MongoDB Atlas...
29
30       at db/conn.mjs:9:11
31
32
33 Test Suites: 2 passed, 2 total
34 Tests:       11 passed, 11 total
35 Snapshots:   0 total
36 Time:        1.121 s
37 Ran all test suites.
38 Force exiting Jest: Have you considered using `--detectOpenHandles` to detect async operations that kept running after all tests finished?

Run actions/setup-node@v4                                                           2s
```

8. Güvenlik

8.1 Pod Güvenlik Bağlamı (securityContext)

Her pod ve konteyner, en az ayrıcalık ilkesine göre yapılandırılmıştır:

Konteyner	runAsUser	readOnlyRootFilesystem	Capabilities	seccompProfile
client (nginx)	101	true	drop ALL	RuntimeDefault
server (node)	1001	true	drop ALL	RuntimeDefault
mongodb	999	false*	drop ALL	RuntimeDefault
python-etl	1001	true	drop ALL	RuntimeDefault

* MongoDB'nin data dizinine yazabilmesi için readOnlyRootFilesystem: false olarak belirlenmiştir.

RuntimeDefault ile dockerın kendi default ayarlarının kullanılması sağlanmıştır. Böylece izinsiz erişim sağlansa bile sistem çekirdeğine çağrı/erişim sağlanamaz.

Capabilities drop ALL belirlenerek, konteyner içinde root userların bile işletim sistemi seviyesinde işlem yapmaları engellenir.

8.2 MongoDB Kimlik Doğrulama

- MONGO_INITDB_ROOT_USERNAME ve MONGO_INITDB_ROOT_PASSWORD ile admin kullanıcı oluşturulur
- Kimlik bilgileri GitHub Secrets'ta saklanır; envsubst ile Kubernetes Secret'a enjekte edilir. Herhangi bir yerde plaintext yada base64 formatında yazılmaz. Doğrudan CI/CD sürecinde github'dan çekilir ve testler sorunsuz tamamlanınca deploymenta bu şekilde gönderilir.

8.3 Ağ Güvenliği

- MongoDB Servisi ClusterIP — cluster dışından erişilemez
- Server Servisi ClusterIP — yalnızca nginx üzerinden erişilir
- ALB Security Group: yalnızca 80/443 dışarıya açık
- Node Security Group: yalnızca ALB'den 8080 ve 5050 portuna izin verildi

8.4 Hassas Bilgilerin Tutulması

- Tüm hassas bilgiler (AWS anahtarları, MongoDB parolası, ECR URL'si) GitHub Secrets'ta
- Kubernetes Secret'lar envsubst ile çalışma zamanında oluşturulur, kod deposunda düz metin yoktur
- ECR imaj taraması (image scanning on push) aktif

9. İzleme ve Alarmlama

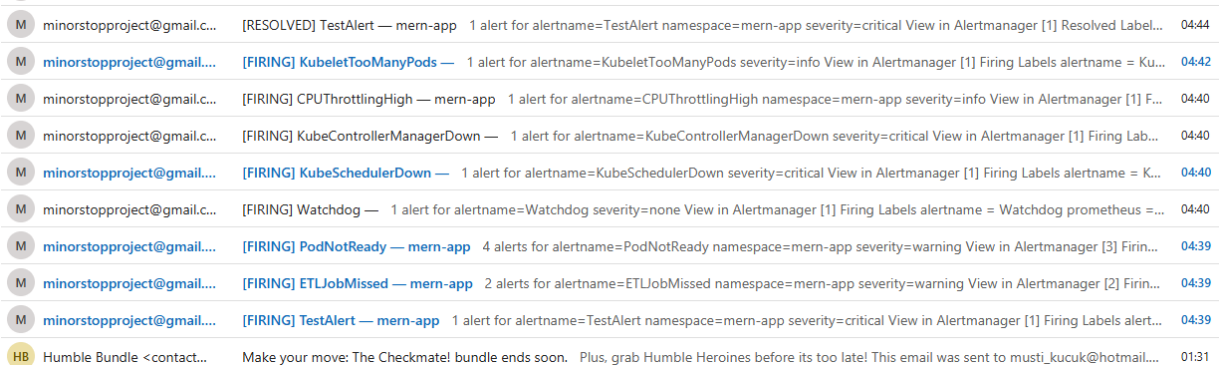
kube-prometheus-stack Helm chart'ı ile Prometheus, Grafana ve Alertmanager monitoring namespace'ine kurulmuştur. Grafana'ya ALB URL'si üzerinden erişilmektedir.

9.1 Alarm Kuralları

Alarm	Tetikleyici	Süre	Önem
HighCPUUsage	CPU > %80 (pod genelinde)	5 dk	warning
PodCrashLooping	CrashLoopBackOff tespit	2 dk	critical
PodNotReady	Pod ready değil	5 dk	warning
HighMemoryUsage	Bellek > %85	10 dk	warning
DeploymentReplicasMismatch	İstenen ≠ hazır replika	5 dk	warning
PersistentVolumeFillingUp	PVC > %85 dolu	10 dk	warning
NodeNotReady	Kubernetes node hazır değil	5 dk	critical

9.2 E-posta Alarm Yapılandırması

- SMTP: smtp.gmail.com:587 (STARTTLS)
- Gönderen: minorstopproject@gmail.com (Gmail uygulama parolası ile)
- Alertmanager, alarm tetiklendiğinde otomatik e-posta gönderir ve çözümlendiğinde bildirir (send_resolved: true)



9.3 Grafana Dashboard'ları

kube-prometheus-stack çok sayıda dashboardu kullanıma hazır olarak vermektedir. Bunlardan bu projede önemli olan ve kullanılabilecek şu şekildedir:

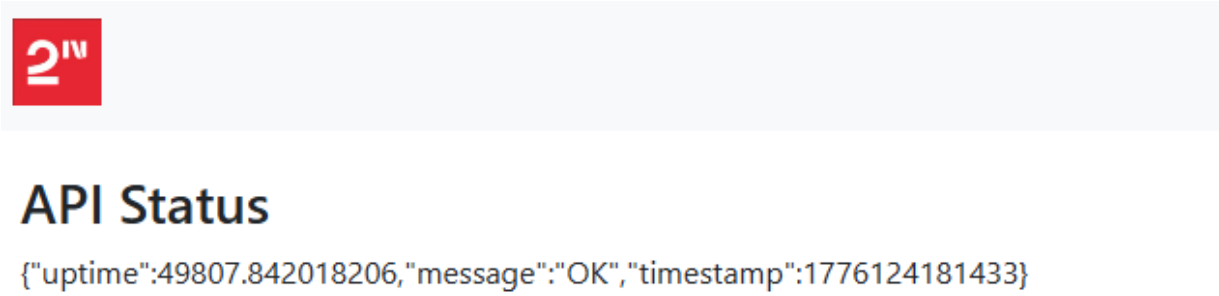
- Kubernetes / Compute Resources / Cluster: Küme genelinde CPU ve bellek kullanımı
- Kubernetes / Compute Resources / Node (Pods): Node başına pod kaynak tüketimi
- Kubernetes / Compute Resources / Namespace (Pods): Namespace bazlı kaynak özeti
- Kubernetes / Compute Resources / Pod: Tek pod detayı
- Kubernetes / Networking / Cluster: Ağ I/O metrikleri
- Kubernetes / Persistent Volumes: PVC kullanım oranları
- Node Exporter / Full: VM düzeyinde CPU, disk, RAM, ağ
- Alertmanager / Overview: Aktif ve sessizleştirilmiş alarmlar

10. Sonuç

Bu proje, iki farklı uygulamanın (MERN stack ve Python ETL) sıfırdan production-grade bir Kubernetes ortamına başarıyla taşındığını göstermektedir. Elde edilen temel kazanımlar:

- Tam otomatik CI/CD: Testler geçmeden deploy edilmez; her commit izlenebilir bir ECR tag'i alır
- Güvenli konteyner mimarisi: Tüm servisler non-root, read-only filesystem, minimum Linux kapasitesi ile çalışır
- Sıfır kesinti deployment: maxUnavailable: 0 stratejisi ile canlı ortamda hizmet kesintisiz güncelleme sağlanır
- Gözlemlenebilirlik: Prometheus metrikleri, Grafana dashboard'ları ve Alertmanager e-posta bildirimleri
- Altyapı kodu: Terraform ile tüm AWS kaynakları tekrar üretilebilir ve versiyonlanabilir
- Maliyet kontrolü: ECR lifecycle policy, HPA ile dinamik ölçekleme, küçük base imajlar

MERN projesinin özette yer alan alan adında ki ekran görüntüleri şu şekildedir



Create New Record

Name

Mustafa Küçükdemirci

Position

DevOps(SRE) Engineer

○ Intern ◉ Junior ○ Senior

Create person

Record List

Name	Position	Level	Action
Mustafa Küçükdemirci	DevOps(SRE) Engineer	Junior	Edit | Delete

Update Record

Name:

Mustafa Küçükdemirci

Position:

DevOps(SRE) Engineer

○ Intern ◉ Junior ○ Senior

Update Record